LatchMoE: Graph-Compatible Expert Offloading through Address-Stable Residency

Anonymous submission

Abstract

Sparse mixture-of-experts (MoE) models reduce arithmetic per token but retain a large parameter footprint. Expert offloading can fit such models on a memory-constrained accelerator, yet conventional offloading changes weight locations on the decode path. This behavior conflicts with graph replay, which relies on stable addresses and a repeatable execution topology; disabling replay exposes a host-bound decode path.

We present LatchMoE, an expert-offloading runtime that separates dynamic expert residency from captured tensor addresses. LatchMoE preallocates fixed device slots, stages routed experts outside the replay boundary, protects slots with an explicit transfer/compute lifecycle, and executes oversized prefill working sets in capacity-bounded waves. The resulting data plane accepts placement plans without embedding a particular prediction policy.

The implementation targets a hook-enabled vLLM-Ascend stack and includes host-side correctness tests and a manifest-oriented online benchmark harness. Existing engineering measurements on Qwen3-30B-A3B report 1.66× lower median time to first token, 2.30× lower median time per output token, and 2.22× higher output throughput than the same offload configuration with graph capture disabled. These numbers are preliminary because their raw run manifests are not yet checked into this repository. We therefore state the mechanism and correctness claims independently and specify the repeated online evaluation required before submission.

1 Introduction

Mixture-of-experts models activate only a small subset of their parameters for each token, but all experts must remain available for routing. This mismatch between sparse computation and dense capacity makes expert weights a dominant memory cost. Parallel systems distribute experts across devices [3], while single-device systems can instead retain inactive experts in host memory and transfer them on demand [5]. The latter approach reduces device-memory demand but adds a transfer decision to the latency-critical serving loop.

Modern serving engines amortize host launch overhead by capturing repeated decode work into an accelerator graph. Expert offloading breaks a hidden assumption of this optimization: the tensors referenced by a replayed graph must have stable addresses even when routing changes which expert is needed. A direct implementation either copies weights into changing allocations inside the captured region or disables capture. The first choice invalidates replay semantics; the second returns decode to a host-dispatched execution path.

The key observation behind LatchMoE is that expert *identity* may change without changing the addresses consumed by the captured kernels. We allocate a fixed bank of expert slots and expose a logical-to-physical mapping to the MoE computation. Routing changes update the mapping and stage weights at a replay boundary; captured kernels continue to read the same slot tensors. This separation turns dynamic residency into metadata and transfer work around an otherwise stable graph.

Address stability alone is insufficient. A transfer must not overwrite a slot still used by in-flight compute, and a prompt can activate more experts than the bank can hold. LatchMoE therefore pairs the slot bank with a checked lifecycle and capacity-bounded wave prefill. Events establish when a slot may be reused, while wave execution limits the simultaneous working set and can overlap staging for the next wave.

We implement LatchMoE as a vLLM platform plugin with a narrow set of vLLM-Ascend hook points. The repository also contains a server-based benchmark that records time to first token (TTFT), time per output token (TPOT), throughput, graph events, HBM use, and transfer events under one artifact contract. We do not treat a forced eager run as a successful graph result; eager execution appears only as an explicit comparison case.

This paper makes three contributions:

- It identifies address instability, rather than expert sparsity alone, as the systems mismatch between offloading and graph replay.
- It introduces an address-stable expert data plane with replay-boundary staging, compute-protected slot reuse, and capacity-bounded wave prefill.
- It defines a reproducible evaluation contract that separates graph correctness, memory feasibility, transfer behavior, and service metrics.

The remainder of the paper describes the mismatch and design requirements (§2), presents the data plane and lifecycle (§3), discusses implementation (§4), and reports current evidence and the submission evaluation plan (§5).

2 Background and Problem

2.1 MoE residency under limited memory

An MoE layer contains E experts but routes each token to only $k \ll E$ of them. Sparse activation reduces compute, as in Switch Transformers [1], but not the capacity needed to make every expert available. Offloading exploits temporal sparsity: inactive weights remain in host memory, and the runtime caches or prefetches experts into a bounded device-resident set. Systems such as MoE-Infinity use activation traces to improve these decisions [5]; FlexGen demonstrates the broader value of coordinating model storage across heterogeneous memory [4].

The runtime cost is determined not only by bytes transferred. Demand loads can stall a decode step, prefetches can contend with computation, and residency updates can add host work. For graph-based execution, residency also interacts with capture validity.

2.2 Why ordinary offloading breaks replay

Graph replay records a repeated sequence of operations and their tensor relationships. A naive expert cache materializes the selected expert in a new buffer or mutates the weight pointer used by the MoE kernel. Both approaches make the repeated decode iteration depend on dynamic host actions inside the captured region. Falling back to eager execution restores flexibility but removes the optimization we seek to preserve.

We require four properties:

1. **Address stability:** captured kernels reference a fixed set of device allocations.
2. **Ownership safety:** a slot has one logical expert and cannot be overwritten while compute uses its current generation.

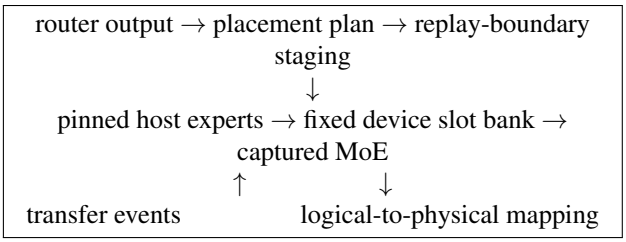

Figure 1: LatchMoE keeps tensor addresses stable and moves dynamic routing into metadata and transfers outside the captured region.

3. **Capacity safety:** no route may silently exceed the configured slot budget.
4. **Observable replay:** graph capture, replay, and fallback events are recorded and cannot be replaced by a forced eager success.

3 Design

3.1 Fixed-slot expert virtualization

At initialization, LatchMoE allocates a fixed number of equal-shaped expert slots. A slot has a stable tensor address, an owner (ℓ, e) consisting of layer and expert identifiers, and a monotonically increasing generation. The logical-to-physical map translates a routed expert to its current slot. Captured grouped matrix multiplications consume slot tensors and indices, not fresh expert allocations.

The slot count is derived from the offload target, physical HBM, a KV-cache reserve, and a minimum net-saving constraint. Explicit capacity checks reject an active working set that cannot be represented by the current execution mode. This prevents an apparently successful run from borrowing unaccounted memory.

3.2 Replay-boundary staging

The router produces active expert identities. A placement-plan adapter computes retain, evict, demand-load, and optional prefetch sets. The adapter is deliberately policy-neutral: LRU or predictive control can be supplied by a separate controller without changing the slot bank.

Staging occurs before the captured MoE region. Host weights are copied on a dedicated transfer stream, and the mapping is published only after the corresponding load completes. The graph then replays against stable slots. No transfer or allocation is hidden inside the captured kernel sequence.

3.3 Compute-protected lifecycle

Each slot progresses through an ownership protocol: free, loading, ready, computing, and eviction-pending. Loading establishes a new owner and generation. Compute records the generation it consumes and protects that slot until its completion event. Eviction waits for both transfer and compute dependencies. Shutdown drains pending work before releasing storage.

The central safety invariant is:

> A mapping may expose expert (ℓ, e) at slot s only if s is ready, owns (ℓ, e), and its published generation equals the generation checked by the consumer.

Violations fail closed rather than selecting an eager path.

3.4 Capacity-bounded wave prefill

Prefill can route many tokens and activate more experts than the slot bank can hold simultaneously. LatchMoE partitions routed work into waves whose expert set fits the bank. For each wave, it stages missing experts, computes the corresponding token subset, and restores output order before combining results. With multiple staging buffers, the next wave's transfers can overlap the current wave's compute.

Wave construction preserves token-to-expert assignments; it changes execution order, not routing semantics. Correctness tests compare the scattered and restored output against the unpartitioned result.

4 Implementation

LatchMoE is a Python package registered through vLLM's platform-plugin entry point. The implementation contains the slot bank, pinned host store, transfer engine, prefill wave scheduler, automatic capacity configuration, profile events, and fused-MoE adapters. A hook-enabled vLLM-Ascend fork exposes the boundaries at which routing, staging, and graph state can be coordinated.

The integration leaves request scheduling and the OpenAI-compatible API unchanged. Disabling the plugin restores the runtime's null hooks. Native weight-offload flags cannot be combined with LatchMoE because two independent owners of expert storage would violate the slot invariant.

5 Evaluation

We separate existing preliminary observations from the experiments required to support a submission claim.

Metric	Capture off	Capture on
Median TTFT (ms)	2283.2	1373.6
Median TPOT (ms/token)	192.5	83.6
Output throughput (token/s)	4.71	10.47

Table 1: Preliminary engineering measurements recorded by the project. Raw manifests and repetitions are required before these values become paper evidence.

5.1 Current evidence

The repository includes host-side tests for slot ownership, phase splitting, wave restoration, graph guards, and benchmark artifact collection. These tests validate contracts but do not measure online performance.

Table 1 reproduces existing engineering measurements from the project record. The configuration is Qwen3-30B-A3B in BF16 on one 64-GiB Ascend 910B-class device, TP1, `max_num_seqs=1`, 200 ShareGPT requests, and a 14-GiB offload budget. Both columns use the LatchMoE offload data plane; only graph capture is toggled. The raw manifest-backed run bundle is not checked into this repository, so the values are preliminary and must not be interpreted as repeated or independently reproduced results.

5.2 Submission experiment matrix

The checked-in harness defines real ShareGPT prompt buckets, explicit graph and eager cases, 14- and 28-GiB budgets, and artifact-preserving failure outcomes. The final evaluation must answer four questions:

Q1: Feasibility. Does full residency fail under the same HBM/KV reservation for which LatchMoE serves requests successfully?

Q2: Graph compatibility. Do capture and replay complete, and what fraction of decode steps replay without fallback?

Q3: Performance. How do TTFT, TPOT, inter-token latency, throughput, and failure rate compare with native offload and the same data plane with capture disabled?

Q4: Attribution. What do fixed slots, asynchronous loading, transfer ordering, wave prefill, and automatic capacity selection each contribute?

Each cell requires three independent service starts and must report median and dispersion. Results must carry parent/runtime revisions, device identity, model and workload digests, graph mode, HBM accounting, raw client metrics, server logs, and transfer profiles. Formal runs may use only an idle device from NPU4–7. A graph failure is retained as a failed artifact, not rerun with forced eager execution.

6 Discussion and Limitations

Policy is intentionally separate. LatchMoE supplies the data plane and a generic plan interface. Predictive placement, route-transition models, and EPLB comparisons belong to a separate control-plane study. This boundary allows all policies to share identical slots, transfers, and graph behavior.

Current scope. The validated configuration is a single low-concurrency device and one unquantized MoE model. Multi-device expert parallelism changes the comparison because its transfers are device-to-device rather than host-to-device. Higher concurrency can also change active-set size and overlap opportunities.

Evidence gap. The absence of checked-in raw artifacts is the primary submission blocker. Until the repeated matrix is complete, the paper supports a mechanism and correctness argument, not a general performance claim.

7 Related Work

MoE architectures use sparse routing to increase parameter capacity without activating every parameter per token [1]. DeepSpeed-MoE combines model and parallelism techniques for large-scale MoE inference [3]. MoE-Infinity focuses on activation-aware caching and prefetch for expert offloading [5]. LatchMoE addresses a different constraint: keeping the offload data plane compatible with captured decode execution through stable slot addresses and an explicit lifecycle.

Heterogeneous inference systems such as FlexGen coordinate storage and execution across device, host, and disk memory [4]. vLLM uses paged KV-cache management and continuous batching to improve LLM serving [2]. LatchMoE is complementary: it virtualizes expert weights rather than KV blocks and preserves the repeated MoE execution graph while residency changes.

8 Conclusion

LatchMoE decouples dynamic expert identity from the stable addresses required by graph replay. Fixed slots, replay-boundary staging, protected ownership, and wave prefill form a graph-compatible offload data plane. The current artifact establishes the design and its host-side contracts; repeated, manifest-backed online evaluation remains necessary to establish the breadth and magnitude of its performance benefit.

References

[1] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120), 2022.

[2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.

[3] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. *arXiv preprint arXiv:2201.05596*, 2022.

[4] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

[5] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. MoE-Infinity: Offloading-efficient MoE model serving. *arXiv preprint arXiv:2401.14361*, 2024.